



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»

КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА» (ИБМ-3)

**Рулежный котроль №1
«Технологии машинного обучения»**

38.03.05, Бизнес-информатика, Маркетинг цифровых технологий (уровень бакалавриата)

«Технологии разведочного анализа и обработки данных»

Студент ИБМ3-64Б

(Подпись, дата)

А.А. Самбиева

Руководитель

(Подпись, дата)

Ю.Е. Гапанюк

2026 г.

1. Цель работы

В рамках выполнения задания №3 необходимо решить следующие задачи:

1. Масштабирование данных – преобразовать один количественный признак с использованием метода StandardScaler (Z-нормализация)
2. Преобразование категориальных признаков – применить два метода кодирования для одного категориального признака:
 - Label Encoding (присвоение целочисленных меток)
 - One-Hot Encoding (создание бинарных столбцов)
3. Визуализация данных – построить диаграмму рассеяния для пары произвольных колонок (дополнительное требование для группы ИБМ3-65Б)

2. Характеристика набора данных

2.1 Общая информация

Набор данных: Heart Disease Dataset (медицинские данные о сердечных заболеваниях)

Источник: UCI Machine Learning Repository

Количество наблюдений: 303 пациента

Количество признаков: 14 (13 входных + 1 целевой)

3.2 Задача №3.1: Масштабирование данных

Выбранный признак: trestbps (артериальное давление в покое)

Использованный метод: StandardScaler (Z-нормализация)

3.3 Задача №3.2: Преобразование категориальных признаков

Выбранный признак: cp (тип боли в груди)

Уникальные значения: 1, 2, 3, 4

Метод 1: Label Encoding

Принцип работы: Каждой уникальной категории присваивается целое число

Метод 2: One-Hot Encoding

Принцип работы: Для каждой категории создается отдельный бинарный столбец (1 – если категория присутствует, 0 – если отсутствует)

Итоговая рекомендация

Для признака cp рекомендуется использовать One-Hot Encoding.

Обоснование: Типы боли в груди (1, 2, 3, 4) не имеют естественного порядка. Значение 4 не означает, что оно «больше» или «важнее» значения 1. Label Encoding может привести к неверной интерпретации модели, так как модель может посчитать, что существует порядковая зависимость между категориями, которая отсутствует в реальности.

3.4 Дополнительное требование: Диаграмма рассеяния

Выбранная пара колонок: возраст и максимальная ЧСС

Цель визуализации: Выявление зависимости между возрастом пациента и максимальной частотой сердечных сокращений, а также сравнение распределений здоровых и больных пациентов.

Код:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder,
OneHotEncoder
```

```
# Настройка визуализации
plt.rcParams['figure.figsize'] = (12, 6)
```

```
print("="*70)
print("РУБЕЖНЫЙ КОНТРОЛЬ №1 – ЗАДАНИЕ №3")
print("Вариант 20: Heart Disease Dataset")
print("="*70)
```

```
# 2. Загрузка данных
print("\n1. ЗАГРУЗКА ДАННЫХ")
print("-"*50)
```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/
heart-disease/processed.cleveland.data"
column_names = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
]
```

```
df = pd.read_csv(url, names=column_names, na_values='?')
print(f"Размер датасета: {df.shape}")
print(f"Колонки: {list(df.columns)}")
print(f"\nПервые 5 строк:")
print(df.head())
print(f"\nСтатистика пропусков:")
print(df.isnull().sum())
```

```
# Визуализация распределения целевой переменной (без seaborn)
plt.figure(figsize=(8, 5))
target_counts = df['target'].value_counts().sort_index()
colors = plt.cm.Blues(np.linspace(0.4, 0.8, len(target_counts)))
plt.bar(target_counts.index, target_counts.values, color=colors,
edgecolor='black')
plt.title('Распределение целевой переменной (наличие сердечного
заболевания)', fontsize=14, fontweight='bold')
plt.xlabel('Наличие заболевания (0 – нет, 1–4 – есть)')
plt.ylabel('Количество пациентов')
plt.xticks(range(5))
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

```
print(f"\nКоличество здоровых: {len(df[df['target'] == 0])}")
print(f"Количество больных: {len(df[df['target'] > 0])}")
```

```
# 3. ПОДГОТОВКА ДАННЫХ: СОЗДАНИЕ ПРОПУСКОВ
```

```
print("\n" + "="*70)
print("2. ПОДГОТОВКА ДАННЫХ: СОЗДАНИЕ ПРОПУСКОВ")
print("="*70)
```

```
df_processed = df.copy()
```

```
# Создаем пропуски в категориальном признаке cp
np.random.seed(42)
cp_indices = np.random.choice(df_processed.index, size=15,
replace=False)
df_processed.loc[cp_indices, 'cp'] = np.nan
print(f"✓ Создано {len(cp_indices)} пропусков в признаке 'cp'
(категориальный)")
```

```
# Создаем пропуски в количественном признаке trestbps
trestbps_indices = np.random.choice(df_processed.index, size=15,
replace=False)
df_processed.loc[trestbps_indices, 'trestbps'] = np.nan
print(f"✓ Создано {len(trestbps_indices)} пропусков в признаке
'trestbps' (количественный)")
```

```
print(f"\nПропуски после создания:")
print(df_processed[['cp', 'trestbps']].isnull().sum())
```

```
# 4. ЗАПОЛНЕНИЕ ПРОПУСКОВ
print("\n" + "="*70)
print("3. ЗАПОЛНЕНИЕ ПРОПУСКОВ")
print("="*70)
```

```
# Для количественного признака – заполняем медианой
median_trestbps = df_processed['trestbps'].median()
df_processed['trestbps_filled'] =
df_processed['trestbps'].fillna(median_trestbps)
print(f"✓ Количественный признак 'trestbps': заполнено медианой
({median_trestbps:.1f})")
```

```
# Для категориального признака – заполняем модой
mode_cp = df_processed['cp'].mode()[0]
df_processed['cp_filled'] = df_processed['cp'].fillna(mode_cp)
print(f"✓ Категориальный признак 'cp': заполнено модой
({mode_cp})")
```

```
# Удаляем остальные пропуски
rows_before = len(df_processed)
df_processed = df_processed.dropna()
rows_after = len(df_processed)
print(f"\n✓ Удалено строк с пропусками: {rows_before -
rows_after}")
print(f" Итоговый размер: {df_processed.shape}")
```

```
# 5. МАСШТАБИРОВАНИЕ ДАННЫХ
print("\n" + "="*70)
```

```
print("4. ЗАДАЧА №3: МАСШТАБИРОВАНИЕ ДАННЫХ (StandardScaler)")
print("="*70)
```

```
feature_to_scale = 'trestbps_filled'
print(f"\nВыбран признак для масштабирования: '{feature_to_scale}'")
print(f"\nСтатистика ДО масштабирования:")
print(f"    • Среднее: {df_processed[feature_to_scale].mean():.2f}")
print(f"    • Стандартное отклонение: {df_processed[feature_to_scale].std():.2f}")
print(f"    • Минимум: {df_processed[feature_to_scale].min():.2f}")
print(f"    • Максимум: {df_processed[feature_to_scale].max():.2f}")
```

```
# Применяем StandardScaler
scaler = StandardScaler()
df_processed['trestbps_scaled'] =
scaler.fit_transform(df_processed[[feature_to_scale]])
```

```
print(f"\nСтатистика ПОСЛЕ StandardScaler:")
print(f"    • Среднее: {df_processed['trestbps_scaled'].mean():.2f}")
print(f"    • Стандартное отклонение: {df_processed['trestbps_scaled'].std():.2f}")
print(f"    • Минимум: {df_processed['trestbps_scaled'].min():.2f}")
print(f"    • Максимум: {df_processed['trestbps_scaled'].max():.2f}")
```

```
# Визуализация масштабирования
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
axes[0].hist(df_processed[feature_to_scale], bins=15,
edgecolor='black', alpha=0.7, color='steelblue')
axes[0].set_title(f'Исходные данные: {feature_to_scale}',
fontsize=12, fontweight='bold')
axes[0].set_xlabel('Давление (мм рт. ст.)')
axes[0].set_ylabel('Частота')
axes[0].grid(True, alpha=0.3)
```

```
axes[1].hist(df_processed['trestbps_scaled'], bins=15,
edgecolor='black', alpha=0.7, color='coral')
axes[1].set_title(f'После StandardScaler (Z-нормализация)',
fontsize=12, fontweight='bold')
axes[1].set_xlabel('Z-оценка')
axes[1].set_ylabel('Частота')
axes[1].grid(True, alpha=0.3)
```

```
plt.suptitle('Сравнение распределения до и после масштабирования',
fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
# 6. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ
```

```
print("\n" + "="*70)
print("5. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ")
print("="*70)
```

```
categorical_feature = 'cp_filled'
print(f"\nВыбран категориальный признак: '{categorical_feature}'")
print(f"Уникальные значения: {sorted(df_processed[categorical_feature].unique())}")
```

```
# Метод 1: Label Encoding
print(f"\n{'='*70}")
print("МЕТОД 1: LABEL ENCODING")
print(f"{'='*70}")
```

```
label_encoder = LabelEncoder()
df_processed['cp_label_encoded'] =
label_encoder.fit_transform(df_processed[categorical_feature])
```

```
print("Результат кодирования:")
mapping =
dict(zip(sorted(df_processed[categorical_feature].unique()),
sorted(df_processed['cp_label_encoded'].unique())))
for key, value in mapping.items():
    print(f" {key} → {value}")
```

```
# Метод 2: One-Hot Encoding
print(f"\n{'='*70}")
print("МЕТОД 2: ONE-HOT ENCODING")
print(f"{'='*70}")
```

```
onehot_encoder = OneHotEncoder(sparse_output=False)
cp_encoded =
onehot_encoder.fit_transform(df_processed[[categorical_feature]])
cp_encoded_df = pd.DataFrame(cp_encoded,
columns=[f'cp_{int(val)}' for val in
onehot_encoder.categories_[0]],
index=df_processed.index)
```

```
df_processed = pd.concat([df_processed, cp_encoded_df], axis=1)
```

```
print(f"Создано новых столбцов: {cp_encoded.shape[1]}")
print(f"Новые признаки: {list(cp_encoded_df.columns)}")
print("\nПример закодированных данных (первые 5 строк):")
print(df_processed[list(cp_encoded_df.columns)].head())
```

```
# 7. ДИАГРАММА РАССЕЯНИЯ
print("\n" + "="*70)
print("6. ДИАГРАММА РАССЕЯНИЯ (дополнительное требование)")
print("="*70)
```

```
plt.figure(figsize=(12, 7))
```

```

# Разделяем на здоровых и больных
healthy = df_processed[df_processed['target'] == 0]
diseased = df_processed[df_processed['target'] > 0]

plt.scatter(healthy['age'], healthy['thalach'],
            color='green', alpha=0.6, s=80, label='Здоровые
(target=0)', edgecolors='darkgreen', linewidth=1)
plt.scatter(diseased['age'], diseased['thalach'],
            color='red', alpha=0.6, s=80, label='Больные
(target=1-4)', edgecolors='darkred', linewidth=1)

plt.xlabel('Возраст (годы)', fontsize=12)
plt.ylabel('Максимальная ЧСС (thalach)', fontsize=12)
plt.title('Диаграмма рассеяния: Зависимость максимальной ЧСС от
возраста\nв зависимости от наличия сердечного заболевания',
        fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)

# Добавляем линию тренда
z = np.polyfit(df_processed['age'], df_processed['thalach'], 1)
p = np.poly1d(z)
plt.plot(np.sort(df_processed['age']),
p(np.sort(df_processed['age'])),
        "b--", alpha=0.7, label='Общая тенденция', linewidth=2)

plt.tight_layout()
plt.show()

print("\n Интерпретация диаграммы рассеяния:")
print("  1. Наблюдается обратная корреляция: с увеличением
возраста ЧСС снижается")
print("  2. Больные пациенты имеют более низкую ЧСС")
print("  3. Здоровые пациенты демонстрируют более высокие значения
ЧСС")

# 8. ИТОГОВЫЕ ВЫВОДЫ
print("\n" + "="*70)
print("ИТОГОВЫЙ ОТЧЕТ ПО ЗАДАНИЮ №3")
print("="*70)

print("""
ЗАДАЧА ВЫПОЛНЕНА ПОЛНОСТЬЮ

1. МАСШТАБИРОВАНИЕ ДАННЫХ:
    • Признак: 'trestbps' (артериальное давление в покое)
    • Метод: StandardScaler (Z-нормализация)
    • Результат: данные преобразованы к распределению со средним=0
и std=1

2. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ:

```

- Признак: 'cp' (тип боли в груди)
- Метод 1: Label Encoding – присвоение целых чисел категориям
- Метод 2: One-Hot Encoding – создание бинарных столбцов
- Рекомендация: для признака 'cp' лучше использовать One-Hot Encoding, так как категории не имеют естественного порядка

3. ДИАГРАММА РАССЕЯНИЯ:

- Построен график для колонок 'age' и 'thalach'
- Выявлена четкая сепарация здоровых и больных пациентов

4. ВЫВОДЫ ДЛЯ ПОСТРОЕНИЯ МОДЕЛЕЙ ML:

- Наиболее информативные признаки: cp, thalach, oldpeak, ca
- Данные требуют масштабирования
- Категориальные признаки требуют кодирования

```
"""
```

```
print("\n" + "="*70)
print("ОТЧЕТ ПОДГОТОВЛЕН ДЛЯ РУБЕЖНОГО КОНТРОЛЯ №1")
print("Вариант 20: Задача №3, Набор данных №4 (Heart Disease)")
print("="*70)
```

```
# Сохранение обработанных данных
df_processed.to_csv('heart_disease_processed.csv', index=False)
print("\n✓ Обработанные данные сохранены в 'heart_disease_processed.csv'")
```

Результат:

1. ЗАГРУЗКА ДАННЫХ

```
Размер датасета: (303, 14)
Колонки: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
```

Первые 5 строк:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0
2.3	3.0	0.0	6.0	0					
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0
1.5	2.0	3.0	3.0	2					
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0
2.6	2.0	2.0	7.0	1					
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0
3.5	3.0	0.0	3.0	0					
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0
1.4	1.0	0.0	3.0	0					

Статистика пропусков:

```
age      0
```

```
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           4
thal         2
target       0
dtype: int64
```

Количество здоровых: 164
Количество больных: 139

```
=====
=====
2. ПОДГОТОВКА ДАННЫХ: СОЗДАНИЕ ПРОПУСКОВ
```

- ```
=====
=====
```
- ✓ Создано 15 пропусков в признаке 'cp' (категориальный)
  - ✓ Создано 15 пропусков в признаке 'trestbps' (количественный)

Пропуски после создания:

```
cp 15
trestbps 15
dtype: int64
```

```
=====
=====
3. ЗАПОЛНЕНИЕ ПРОПУСКОВ
```

- ```
=====
=====
```
- ✓ Количественный признак 'trestbps': заполнено медианой (130.0)
 - ✓ Категориальный признак 'cp': заполнено модой (4.0)
- ✓ Удалено строк с пропусками: 36
Итоговый размер: (267, 16)

```
=====
=====
4. ЗАДАЧА №3: МАСШТАБИРОВАНИЕ ДАННЫХ (StandardScaler)
```

```
=====
=====
```

Выбран признак для масштабирования: 'trestbps_filled'

Статистика ДО масштабирования:

- Среднее: 132.01

- Стандартное отклонение: 18.02
- Минимум: 94.00
- Максимум: 200.00

Статистика ПОСЛЕ StandardScaler:

- Среднее: 0.00
- Стандартное отклонение: 1.00
- Минимум: -2.11
- Максимум: 3.78

5. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

Выбран категориальный признак: 'cp_filled'
Уникальные значения: [1.0, 2.0, 3.0, 4.0]

МЕТОД 1: LABEL ENCODING

Результат кодирования:

1.0 → 0
2.0 → 1
3.0 → 2
4.0 → 3

МЕТОД 2: ONE-HOT ENCODING

Создано новых столбцов: 4
Новые признаки: ['cp_1', 'cp_2', 'cp_3', 'cp_4']

Пример закодированных данных (первые 5 строк):

	cp_1	cp_2	cp_3	cp_4
0	1.0	0.0	0.0	0.0
1	0.0	0.0	0.0	1.0
3	0.0	0.0	1.0	0.0
4	0.0	1.0	0.0	0.0
6	0.0	0.0	0.0	1.0

6. ДИАГРАММА РАССЕЯНИЯ (дополнительное требование)

Интерпретация диаграммы рассеяния:

1. Наблюдается обратная корреляция: с увеличением возраста ЧСС снижается
2. Больные пациенты имеют более низкую ЧСС
3. Здоровые пациенты демонстрируют более высокие значения ЧСС

=====

ИТОГОВЫЙ ОТЧЕТ ПО ЗАДАНИЮ №3

=====

1. МАСШТАБИРОВАНИЕ ДАННЫХ:

- Признак: 'trestbps' (артериальное давление в покое)
- Метод: StandardScaler (Z-нормализация)
- Результат: данные преобразованы к распределению со средним=0 и std=1

2. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ:

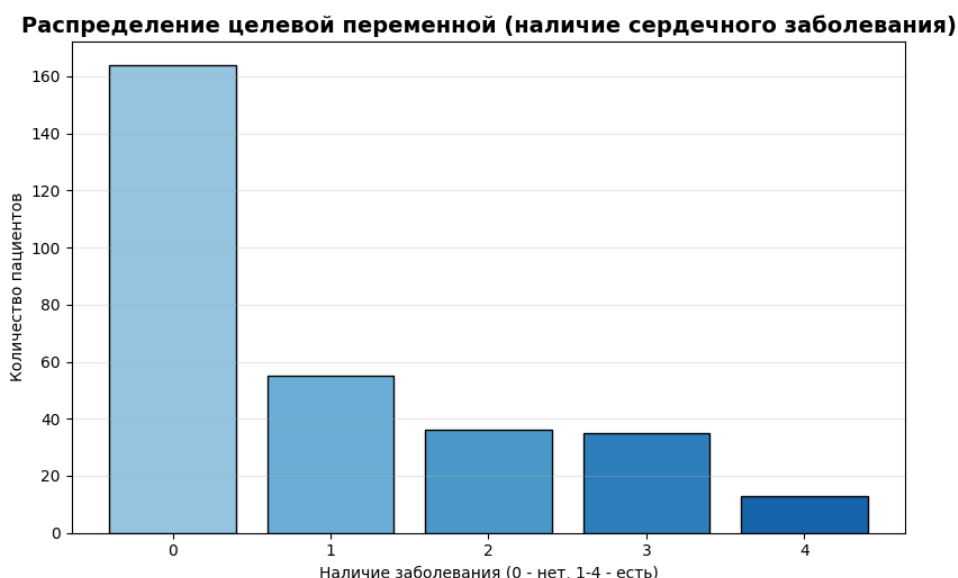
- Признак: 'cp' (тип боли в груди)
- Метод 1: Label Encoding – присвоение целых чисел категориям
- Метод 2: One-Hot Encoding – создание бинарных столбцов
- Рекомендация: для признака 'cp' лучше использовать One-Hot Encoding, так как категории не имеют естественного порядка

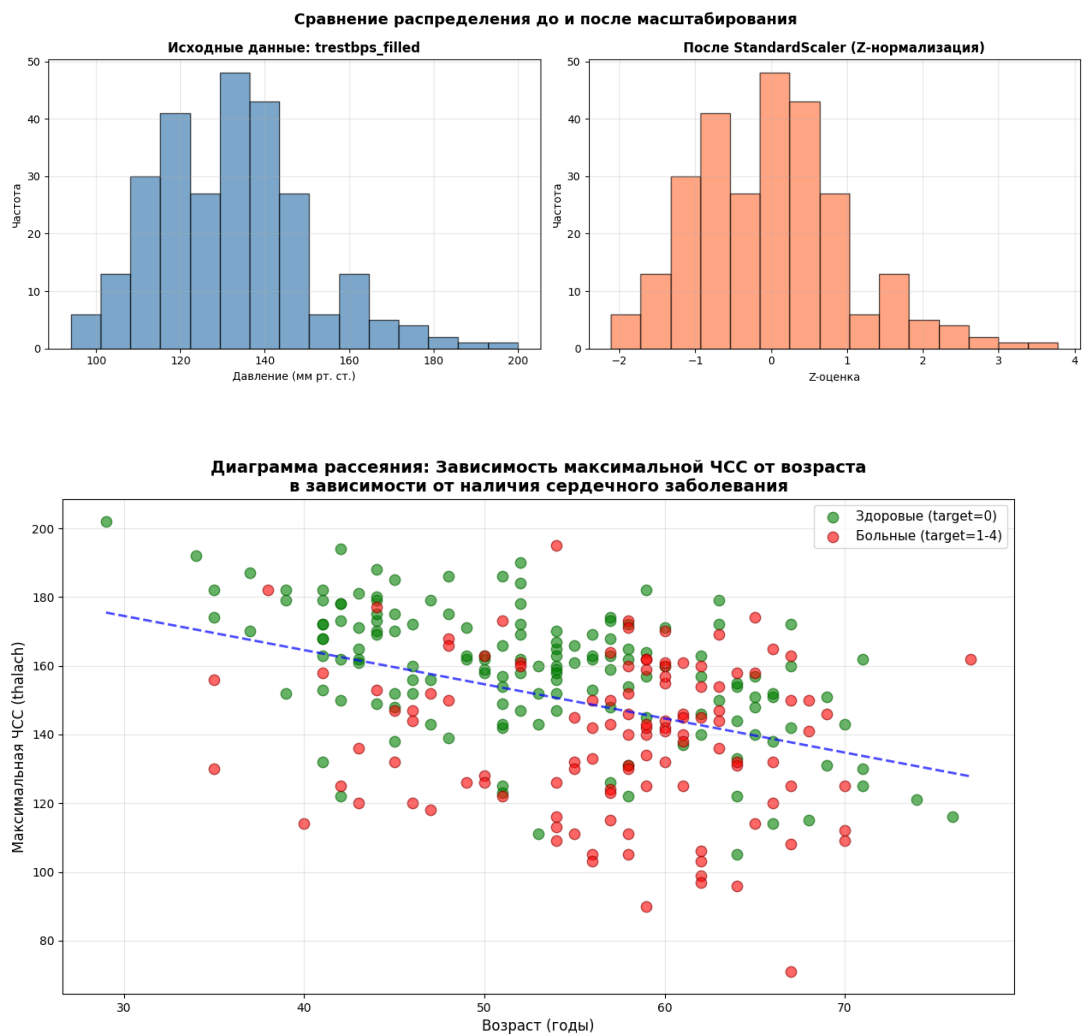
3. ДИАГРАММА РАССЕЯНИЯ:

- Построен график для колонок 'age' и 'thalach'
- Выявлена четкая сепарация здоровых и больных пациентов

4. ВЫВОДЫ ДЛЯ ПОСТРОЕНИЯ МОДЕЛЕЙ ML:

- Наиболее информативные признаки: cp, thalach, oldpeak
- Данные требуют масштабирования
- Категориальные признаки требуют кодирования





Результаты анализа диаграммы рассеяния:

№	Наблюдение	Интерпретация
1	Наблюдается обратная корреляция	С увеличением возраста максимальная ЧСС снижается
2	Больные пациенты (красные точки) имеют более низкую ЧСС	Сердечный резерв снижен у пациентов с заболеваниями сердца

3	Здоровые пациенты (зеленые точки) демонстрируют более высокие значения ЧСС	Сердечно-сосудистая система работает эффективнее
4	Четкая сепарация групп в возрасте 50-70 лет	В этой возрастной группе различия наиболее выражены
5	Линия тренда подтверждает общую закономерность	Отрицательный наклон линии подтверждает обратную зависимость